

TÀI LIỆU KỸ THUẬT GIẢI THUẬT VÀ MÔ HÌNH TOÁN HỌC

Hệ thống giao dịch trao đổi hàng hóa sinh viên O2O & AI (HUNRE E-Commerce)

Đơn vị phát triển: Nhóm Nghiên cứu & Phát triển Hệ thống HUNRE E-Commerce

Cơ sở đào tạo: Khoa Công nghệ Thông tin — Trường Đại học Tài nguyên và Môi trường Hà Nội

Phiên bản tài liệu: 2.1.0 | Phạm vi: Kiến trúc thuật toán lõi trong 5 Microservices và 2 Web Portals

Mục đích: Tài liệu kỹ thuật chi tiết phục vụ thẩm định giải thuật, đối chiếu mã nguồn và nghiệm thu hệ thống.

1. BÀI TOÁN GHÉP CẶP CHU TRÌNH TRAO ĐỔI ĐỒ 3 CHIỀU & BẢO TOÀN SỐ DƯ (AI BARTER GRAPH 2.0)

1.1. Chức năng tương ứng trong hệ thống

- Phân hệ phụ trách: AI Engine Microservice (Port 8005).
- Tập mã nguồn: services/ai-engine/app/services/graph_solver.py.
- Endpoint API: POST /api/v1/ai/barter/solve-cycles.
- Giao diện hiển thị: frontend/student-portal/pages/barter-graph.html (Canvas đồ thị và bảng bù trừ).

1.2. Phát biểu bài toán

Trong kinh tế chia sẻ tại trường đại học, một sinh viên sở hữu vật phẩm u muốn đổi lấy vật phẩm v, nhưng người giữ v không cần u mà lại cần vật phẩm w của sinh viên thứ ba. Giao dịch song phương (2 bên) lập tức bế tắc. Mục tiêu của bài toán là tìm chu trình trao đổi khép kín đa phương $S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow \dots \rightarrow S_k \rightarrow S_1$ (độ dài $k \in [2, 4]$) và giải bài toán bù trừ giá trị chênh lệch tiền mặt để bảo toàn số dư.

1.3. Biểu diễn toán học đồ thị có hướng có trọng số

Định nghĩa đồ thị có hướng $G = (V, E, W)$:

- Tập đỉnh V: Mỗi đỉnh v_i đại diện cho một vật phẩm đăng ký trao đổi. Thuộc tính đỉnh gồm: Owner(v_i) (mã sinh viên), Val(v_i) (giá trị định giá theo VNĐ), Wish(v_i) (tập từ khóa nhu cầu mong muốn nhận lại).
- Tập cạnh E: Cạnh có hướng $(u, v) \in E$ tồn tại khi và chỉ khi:

$$Owner(u) \neq Owner(v) \wedge [Title(v) \wedge Wish(u) \wedge Category(v) \wedge Wish(u) \wedge AcceptAny(u) = True]$$

- Hàm trọng số W: $W(u, v)$ thể hiện độ chênh lệch giá trị giữa vật phẩm nhận về và vật phẩm chuyển giao: $W(u, v) = Val(v) - Val(u)$.

1.4. Thuật toán tìm chu trình Johnson (Johnson's Simple Directed Cycle Finding)

Áp dụng thuật toán Johnson (1975) dựa trên duyệt theo chiều sâu (DFS) có kiểm soát danh sách chặn (Blocked Set) để tìm toàn bộ chu trình đơn có độ dài từ 2 đến L_{max} (mặc định $L_{max} = 3$):

Mã giả Thuật toán Johnson's Cycle Finding kết hợp Ràng buộc Chiều dài:

Đầu vào: Đồ thị có hướng $G = (V, E)$, giới hạn chiều dài $L_{\max} = 3$
Đầu ra: Danh sách các chu trình thỏa mãn $Cycles = \{C_1, C_2, \dots\}$

- Phân rã G thành các thành phần liên thông mạnh (Strongly Connected Components - SCC).
- Với mỗi thành phần SCC k có từ 2 đỉnh trở lên:
 - Chọn đỉnh s có chỉ số thứ tự nhỏ nhất làm đỉnh gốc.
 - Khởi tạo mảng $blocked = [False] * |V|$, $B = \{v: [] \text{ cho mọi } v \text{ trong } V\}$, $stack = []$
 - Hàm đệ quy $Circuit(v)$:
 $f = False$; push v vào $stack$; $blocked[v] = True$
Duyệt từng đỉnh kề w thuộc danh sách kề $Adj[v]$:
Nếu $w == s$:
Nếu $2 \leq \text{độ dài } stack \leq L_{\max}$: Lưu lại chu trình $stack$; $f = True$
Ngược lại nếu $blocked[w] == False$:
Nếu $Circuit(w) == True$: $f = True$
Nếu $f == True$: $Unblock(v)$
Ngược lại: Với mỗi w trong $Adj[v]$, thêm v vào $B[w]$
Pop v khỏi $stack$; trả về f
 - Gọi $Circuit(s)$, sau đó xóa bỏ s khỏi G và lặp lại bước 2.

1.5. Định lý bảo toàn số dư tiền mặt (Zero-Sum Balance Invariant)

Định lý: Xét chu trình $C = (v_1, v_2, \dots, v_k)$. Người sở hữu vật phẩm v_i chuyển giao v_i và nhận về vật phẩm v_{prev} (với $\text{prev} = (i - 1 + k) \bmod k$). Khoản tiền chênh lệch Δ_i được xác định:

$$\Delta_i = Val(v_{\text{prev}}) - Val(v_i)$$

- Nếu $\Delta_i > 0$: Sinh viên nhận đồ có giá trị cao hơn đồ cho đi $\rightarrow$ Phải nộp bù Δ_i vào quỹ ký quỹ Smart Escrow.
- Nếu $\Delta_i < 0$: Sinh viên nhận đồ có giá trị thấp hơn $\rightarrow$ Được nhận lại $|\Delta_i|$ từ quỹ ký quỹ Smart Escrow.
- Nếu $\Delta_i = 0$: Trao đổi ngang giá hoàn hảo.

Tổng dòng tiền thanh toán bù trừ trong chu trình luôn bảo toàn triệt để:

$$\sum_{i=1..k} \Delta_i = \sum_{i=1..k} [Val(v_{\text{prev}}) - Val(v_i)] = \sum_{i=1..k} Val(v_{\text{prev}}) - \sum_{i=1..k} Val(v_i) = 0$$

Ý nghĩa: Tổng tiền nộp bù của các sinh viên nhận đồ đắt hơn bằng chính xác tổng tiền chi trả cho các sinh viên nhận đồ rẻ hơn. Quỹ trung gian Escrow luôn cân bằng tài chính 100%, không phát sinh rủi ro thâm hụt.

Thành viên	Món đồ trao đi	Món đồ nhận về	Chênh lệch Δ_i	Nghiệp vụ Smart Escrow
Nguyễn Văn An	Giáo trình CSDL (75.000đ)	Bàn phím DareU (250.000đ)	+175.000đ	Nộp bù 175.000đ vào Escrow
Trần Thị Bích	Casio FX 580VN (320.000đ)	Giáo trình CSDL (75.000đ)	-245.000đ	Nhận lại 245.000đ từ Escrow
Lê Hoàng Cường	Bàn phím DareU (250.000đ)	Casio FX 580VN (320.000đ)	+70.000đ	Nộp bù 70.000đ vào Escrow
TỔNG CỘNG	645.000đ	645.000đ	0 VNĐ	Bảo toàn dòng tiền tuyệt đối

2. GIẢI THUẬT THẨM ĐỊNH ĐỘ HAO MÒN & PHÂN HẠNG CHẤT LƯỢNG (COMPUTER VISION)

2.1. Chức năng tương ứng trong hệ thống

- Phân hệ phụ trách: AI Engine Microservice.
- Tập mã nguồn: `services/ai-engine/app/services/cv_service.py`.
- Endpoint API: `POST /api/v1/ai/cv/inspect`.
- Giao diện hiển thị: Modal Thẩm định chất lượng khi sinh viên đăng bài tại `frontend/student-portal/index.html`.

2.2. Biểu diễn toán học xử lý tín hiệu hình ảnh

Ảnh đầu vào được chuẩn hóa sang không gian độ xám ma trận $I_{gray}(x, y)$ kích thước $W \times H$:

1. Gradient vi phân không gian 2 chiều: Tính đạo hàm riêng theo trục ngang g_x và trục dọc g_y :

$g_x(x, y) = \partial I_{gray} / \partial x, \quad g_y(x, y) = \partial I_{gray} / \partial y$

2. Độ lớn Gradient tại mỗi điểm ảnh và điểm sắc nét trung bình toàn khung ảnh:

$||\nabla I(x, y)|| = \sqrt{g_x^2(x, y) + g_y^2(x, y)}, \quad S_{sharp} = (1 / (W \times H)) \times \sum_{x, y} ||\nabla I(x, y)||$

3. Chỉ số bất đồng nhất màu sắc (đo lường vết ố vàng / trầy xước):

$\sigma_{color} = (1 / 3) \times \sum_c \sqrt{[(1 / (W \times H)) \times \sum_{x, y} (I(x, y, c) - \mu_c)^2]}$

4. Hàm ước lượng tỷ lệ khiếm khuyết bề mặt:

$D_{ratio} = \min(0.35, \max(0.01, (100 - \min(3 \times S_{sharp}, 90)) / 250))$

2.3. Bảng quy tắc phân lớp nhãn chất lượng (Condition Grade)

Tỷ lệ khiếm khuyết D_{ratio}	Phân Hạng AI	Độ mới thực tế	Mô tả đánh giá chất lượng	Hệ số giá sàn đề xuất
$D_{ratio} < 0.03$	GRADE_S	98% - 99%	Sản phẩm như mới, góc cạnh nguyên vẹn, không vết xước.	90% giá gốc
$0.03 \leq D_{ratio} < 0.08$	GRADE_A	90% - 95%	Tình trạng rất tốt, chỉ có xước dăm cực nhỏ, trang sách sạch.	75% giá gốc
$0.08 \leq D_{ratio} < 0.15$	GRADE_B	80% - 89%	Có dấu vết sử dụng thực tế (bóng phím, sòn mép), hoạt động tốt.	55% giá gốc
$D_{ratio} \geq 0.15$	GRADE_C	< 80%	Nhiều vết xước sâu hoặc viết bút dạ nhiều, mua dùng tạm thời.	35% giá gốc

3. THUẬT TOÁN BẮM CẢM NHẬN THỊ GIÁC (dHash) PHÁT HIỆN ẢNH
MẠNG CHỐNG GIAN LẬN

3.1. Chức năng tương ứng trong hệ thống

- Phân hệ phụ trách: AI Engine Microservice.
- Tệp mã nguồn: services/ai-engine/app/services/anti_fraud.py.
- Endpoint API: POST /api/v1/ai/anti-fraud/check.
- Chức năng: Tự động phát hiện ảnh tải từ Shopee, Lazada, Tiki hoặc ảnh chụp màn hình đăng bán ảo.

3.2. Thuật toán Difference Hashing 64-bit

- Thu nhỏ kích thước (Downsampling): Thu nhỏ ảnh về kích thước cố định $(N + 1) \times N$ với $N = 8$, tức ma trận $9 \times 8 = 72$ pixels bằng thuật toán nội suy Lanczos.
- Chuyển đổi sang ảnh xám (Grayscale): Loại bỏ độ chói màu sắc, giữ lại cường độ sáng.
- So sánh vi phân gradient hàng ngang: So sánh độ sáng giữa 2 điểm ảnh liền kề theo từng hàng:

$B(r, c) = 1 \text{ nếu } P(r, c) > P(r, c + 1); \quad B(r, c) = 0 \text{ nếu } P(r, c) \leq P(r, c + 1) \quad (\text{với } r \in [0, 7], c \in [0, 7])$

- Tạo chữ ký số Hexadecimal 16 ký tự: 64 bit nhị phân được gom thành 8 bytes và chuyển thành chuỗi 16 ký tự Hex.
- Khoảng cách Hamming và nhận dạng ảnh catalog: Độ sai khác giữa 2 ảnh được đo bằng khoảng cách Hamming: $D_H(H_1, H_2) = \sum_{k=1..64} (b_{1,k} \neq b_{2,k})$. Nếu $D_H \leq 5$, hai ảnh được xác định là trùng lặp. Đồng thời, bộ lọc phát hiện tỷ lệ kích thước catalog vuông chuẩn (800×800, 1000×1000, 1200×1200) cảnh báo yêu cầu sinh viên chụp ảnh thực tế.

4. MÔ HÌNH SUY GIẢM HÀM MŨ ĐỊNH GIÁ TỰ ĐỘNG THEO THỜI GIAN (TIME-DECAY PRICING)

4.1. Chức năng tương ứng trong hệ thống

- Phân hệ phụ trách: Product Service (Cronjob) và AI Engine (API).
- Tập mã nguồn: services/product-service/app/Services/TimeDecayPricingService.php và price_agent.py.
- Endpoint API: POST /api/v1/ai/pricing/decay.
- Chức năng: Tự động giảm giá đồ dùng học tập cần thanh lý gấp cuối kỳ theo thời gian thực mà không bao giờ dưới Giá sàn.

4.2. Xây dựng công thức toán học phân rã giá trị

Hệ thống mô hình hóa quá trình giảm giá theo hàm phân rã liên tục với chu kỳ bán rã (Exponential Half-Life Decay):

$$P(t) = P_{\text{floor}} + (P_{\text{orig}} - P_{\text{floor}}) \times e^{-\lambda t}$$

Trong đó:

- $P(t)$: Giá niêm yết tại thời điểm t giờ sau khi đăng bán.
- P_{orig} : Mức giá khởi điểm ban đầu do sinh viên niêm yết.
- P_{floor} : Mức giá sàn tối thiểu do người bán thiết lập để bảo toàn vốn.
- $t_{1/2}$: Thời gian bán rã mặc định là 72 giờ (3 ngày). Hằng số suy giảm $\lambda = \ln(2) / t_{1/2} \approx 0.009627$.
- Bảo đảm bất biến biên (Boundary Invariant): Khi $t \rightarrow \infty$, $P(t) \rightarrow P_{\text{floor}}$ và $\forall t \geq 0, P(t) \geq P_{\text{floor}}$.

5. GIẢI THUẬT ĐÀM PHÁN THƯƠNG LƯỢNG GIÁ ĐỘNG DỰA TRÊN ĐIỂM UY TÍN (AI PRICE NEGOTIATOR)

5.1. Chức năng tương ứng trong hệ thống

- Phân hệ phụ trách: AI Engine Microservice.
- Tập mã nguồn: services/ai-engine/app/services/price_agent.py.
- Endpoint API: POST /api/v1/ai/pricing/negotiate.
- Giao diện hiển thị: Modal "Trà Giá AI" tại trang chủ chợ sinh viên frontend/student-portal/index.html.

5.2. Mô hình cây quyết định đa tiêu chí

Cho người mua đề xuất mức giá P_{offer} và có Điểm Uy Tín $T \in [0, 1000]$. Giá niêm yết hiện tại là P_{curr} và giá sàn người bán là P_{floor} . Tỷ số đề xuất giá tương đối và ngưỡng chấp nhận linh hoạt được xác định:

$$\rho = (P_{\text{offer}} - P_{\text{floor}}) / (P_{\text{curr}} - P_{\text{floor}} + 10^{-5}), \quad \theta_{\text{accept}}(T) = 0.5 - (T / 2000)$$

Quy tắc ra quyết định tự động của AI Agent:

- Nếu $P_{\text{offer}} \geq P_{\text{curr}}$: Chấp thuận ngay lập tức (DECISION = ACCEPT).
- Nếu $P_{\text{offer}} < P_{\text{floor}}$: Mức giá dưới giá sàn. Nếu $T \geq 600$ (Hạng Vàng/Kim Cương), AI đưa ra mức giá thỏa hiệp ưu đãi $P_{\text{counter}} = P_{\text{floor}} \times 1.03$ (COUNTER_OFFER); ngược lại từ chối (REJECT).
- Nếu $P_{\text{floor}} \leq P_{\text{offer}} < P_{\text{curr}}$: Nếu $\rho \geq \theta_{\text{accept}}(T) \rightarrow \text{ACCEPT}$; ngược lại đề xuất giá trung gian $P_{\text{counter}} = \text{Round}((P_{\text{offer}} + P_{\text{curr}}) / 2, -3)$.

6. THUẬT TOÁN XÁC THỰC MÃ QR ĐỘNG DYNAMIC QR TOTP XOAY VÒNG 30S (HMAC-SHA256)

6.1. Chức năng tương ứng trong hệ thống

- Phân hệ phụ trách: O2O Hub Logistics Service (Port 8004).
- Tập mã nguồn: services/hub-service/app/Services/DynamicQREngine.php.
- Endpoint API: GET /api/v1/hub/lockers.
- Giao diện hiển thị: Mã QR xoay vòng 30s tại pages/escrow-order.html và máy quét PWA của thủ kho tại frontend/hub-staff-portal/index.html.

6.2. Cơ chế mật mã học HMAC-SHA256 theo cửa sổ thời gian (RFC 6238)

Nhằm ngăn chặn triệt để hành vi chụp trộm màn hình để lấy đồ tại Trạm Hub, mã QR được tạo động theo chu kỳ 30 giây:

1. Cửa sổ thời gian (Time Window): $W(t) = \lfloor t / T_{\text{step}} \rfloor$ với $T_{\text{step}} = 30$ giây.

2. Chuỗi thông điệp xác thực:

$$M(t) = \text{OrderCode} || \text{UserId} || \text{Role} || W(t)$$

Trong đó: Ký hiệu $||$ là phép nối chuỗi dữ liệu (Concatenation).

3. Chữ ký số xác nhận:

$$\tau(t) = \text{Truncate}_{16}(\text{HMAC-SHA256}(K_{\text{secret}}, M(t)))$$

4. Cơ chế xác thực chịu lỗi trễ đồng hồ: Máy quét tại Trạm Hub giải mã và kiểm tra $\tau(t)$ với $W(t)$ và $W(t) - 1$ (cửa sổ trễ 30 giây) thông qua hàm so sánh bất biến thời gian `hash_equals()` nhằm chống lại tấn công vét cạn (Timing Attack).

7. MÔ HÌNH MÁY TRẠNG THÁI HỮU HẠN & ĐIỀU PHỐI GIAO DỊCH PHÂN TÁN SAGA (SMART ESCROW)

7.1. Chức năng tương ứng trong hệ thống

- Phân hệ phụ trách: Escrow & Payment Microservice (Port 8003).
- Tập mã nguồn: services/escrow-service/app/Services/EscrowSagaOrchestrator.php.
- Endpoint API: GET /api/v1/escrow/transactions, POST /api/v1/escrow/saga/advance.
- Giao diện hiển thị: Thanh tiến trình Stepper 7 bước tại frontend/student-portal/pages/escrow-order.html.

7.2. Định nghĩa máy trạng thái hữu hạn Automaton 7 trạng thái

Tiến trình ký quỹ được mô hình hóa thành một bộ máy trạng thái hữu hạn xác định $M = (S, \Sigma, \delta, s_0, F)$:

- Tập 7 trạng thái: $S = \{\text{INITIATED}, \text{ESCROW_LOCKED}, \text{STORED_AT_HUB}, \text{INSPECTING}, \text{RELEASED}, \text{DISPUTED}, \text{REFUNDED}\}$.
- Trạng thái khởi đầu: $s_0 = \text{INITIATED}$. | Tập trạng thái kết thúc: $F = \{\text{RELEASED}, \text{REFUNDED}\}$.

Trạng thái hiện tại	Sự kiện kích hoạt	Điều kiện kiểm tra Guard	Trạng thái kế tiếp	Hành động Saga đền bù
INITIATED	PAY_DEPOSIT	Tiền vào tài khoản trung gian	ESCROW_LOCKED	Hủy đơn, hoàn cọc
ESCROW_LOCKED	SELLER_CHECKIN	Thủ kho quét mã QR hợp lệ	STORED_AT_HUB	Mở tủ trả đồ về người bán
STORED_AT_HUB	BUYER_CHECKOUT	Người mua quét mã nhận đồ	INSPECTING	Khóa tủ giữ nguyên vị trí

INSPECTING	BUYER_SATISFIED	Người mua bấm xác nhận hài lòng	RELEASED	Giải ngân tiền cho người bán
INSPECTING	RAISE_DISPUTE	Hàng vỡ hỏng / sai mô tả tại Hub	DISPUTED	Lập biên bản trung gian
DISPUTED	ADMIN_RESOLVE	Thủ kho xác nhận hàng không đạt	REFUNDED	Hoàn tiền 100% người mua

8. GIẢI THUẬT ĐÁNH GIÁ TÍN NHIỆM & TÍNH ĐIỂM UY TÍN SINH VIÊN (TRUST SCORE ENGINE)

8.1. Chức năng tương ứng trong hệ thống

- Phân hệ phụ trách: Auth & Identity Microservice (Port 8001).
- Tệp mã nguồn: services/auth-service/app/Services/TrustScoreService.php.
- Endpoint API: GET /api/v1/auth/users/trust-score.
- Giao diện hiển thị: Huy hiệu Điểm Uy Tín gắn liền tài khoản sinh viên trên thanh Navbar toàn hệ thống.

8.2. Hàm tích lũy điểm hành vi rời rạc có chặn biên

Điểm uy tín của sinh viên tại thời điểm $n + 1$ là một hàm tích lũy có chặn trên và chặn dưới trong không gian $T \in [0, 1000]$:

$$T_{n+1} = \max(0, \min(1000, T_n + \sum_j w_j \times E_j))$$

Trong đó các trọng số biến cố hành vi được định lượng cụ thể:

- $w_1 = +5$: Hoàn tất giao dịch O2O đúng hẹn tại Trạm Hub CS1.
- $w_2 = +2$: Nhận đánh giá 5 sao từ sinh viên đối tác.
- $w_3 = -15$: Quá hạn 48h không mang đồ đến gửi tại tủ Trạm Hub.
- $w_4 = -20$: Tự ý hủy kèo sau khi Barter Graph đã ghép cặp thành công.
- $w_5 = -30$: Cố tình khai gian tình trạng sản phẩm bị camera Hub phát hiện.

Thang Điểm Uy Tín	Phân Hạng Sinh Viên (Tier)	Quyền Lợi & Đặc Quyền Trong Hệ Thống
800 - 1000 Điểm	KIM CƯƠNG	Bảo chứng uy tín 100%, được phép nhận hàng trước không cần cọc tiền.
600 - 799 Điểm	VÀNG	Ưu tiên số 1 khi quét chu trình Barter Graph, giảm 50% tiền cọc Escrow.
400 - 599 Điểm	BẠC (Tiêu chuẩn)	Hưởng đầy đủ tính năng mua bán, đổi đồ và trả giá AI thông thường.
200 - 399 Điểm	ĐỒNG	Phải đặt cọc 100% giá trị món đồ khi tham gia ký quỹ tại Trạm Hub.
< 200 Điểm	CẢNH BÁO	Hạn chế quyền đổi đồ, bắt buộc xác minh lại thẻ sinh viên tại Đoàn Trường.

9. BẢNG TỔNG HỢP ẢNH XẠ GIẢI THUẬT & PHÂN HỆ CHỨC NĂNG

S T T	Tên Thuật Toán / Mô Hình	Microservice Phụ Trách	API Endpoint Cung Cấp	Chức Năng Nghiệp Vụ
1	Johnson's Directed Cycle & Zero-Sum	ai-engine (Port 8005)	POST /api/v1/ai/barter/solve-cycles	Ghép vòng tròn đổi đồ 3 chiều và tính toán bù trừ tiền mặt
2	Spatial Gradient & HSV Color Defect	ai-engine (Port 8005)	POST /api/v1/ai/cv/inspect	Thị giác máy tính đo độ hao mòn, phân hạng Grade S/A/B/C
3	Difference Hashing (dHash 64-bit)	ai-engine (Port 8005)	POST /api/v1/ai/anti-fraud/check	Chống ảnh mạng catalog Shopee/Lazada, bảo vệ giao dịch
4	Exponential Half-Life Decay Pricing	product-service (8002)	POST /api/v1/ai/pricing/decay	Tự động giảm giá theo giờ thanh lý giáo trình cuối kỳ
5	Multi-Criteria Price Negotiator Agent	ai-engine (Port 8005)	POST /api/v1/ai/pricing/negotiate	Trợ lý AI tự động đàm phán giá theo Điểm Uy Tín 24/7
6	HMAC-SHA256 Dynamic QR TOTP (30s)	hub-service (Port 8004)	GET /api/v1/hub/lockers	Sinh và giải mã QR xoay 30s chống chụp trộm màn hình
7	Distributed Saga 7-State FSM	escrow-service (8003)	POST /api/v1/escrow/saga/advance	Đóng băng tiền cọc ký quỹ và giải ngân khi hài lòng
8	Accumulative Trust Score Engine	auth-service (Port 8001)	GET /api/v1/auth/users/trust-score	Đánh giá tín nhiệm sinh viên HUNRE và phân cấp đặc quyền